

The Waterfall model becomes a challenge to implement and can cause extensive and expensive rework under scenarios where:

- the requirements are poorly known,
- the requirements are likely to evolve or change, and/or
- it is possible that new requirements emerge during development.

Any issues discovered during later activities such as testing and/or integration can mean a significant amount of rework, since the team may need to go all the way back to requirements and redesign the system. Such rework usually causes significant budget overruns and schedule slips. Furthermore, the Waterfall model does not explicitly incorporate stakeholder feedback during the lifecycle, since the activity is timeboxed within its own stage, which limits its ability to validate concepts before doing too much work or to adapt to changing needs. Moreover, the tightness of the development process eliminates any opportunity for incrementally deploying the system. These aspects make the Waterfall model unattractive for most engineering projects.

2.2. Vee model

The systems engineering Vee model is a plan-driven model that leverages the hierarchical nature of most complex systems. Development occurs in a top-down fashion in which a system is recursively decomposed into lower-level systems (which may be assigned names as subsystems, components, etc.). Importantly, the Vee model recognizes uncertainty and technical maturity are not uniform across all elements of the system, and hence allows for different system elements to evolve at different paces. This is actually its key tenet, that while all activities must end in sequence (as was the case for the Waterfall model), activities may begin in any order. That is, within the Vee model, the activities might not begin in the same order for every system element, but their completion does

occur per the constraints of the process such that integration cannot be completed until all system elements are manufactured. In fact, the Vee model emphasizes the validation and verification at each level and between levels, which can quickly lead to iteration as the team resolves any issues [32] to reduce the financial and schedule impacts of rework. For example, the Vee model encourages to begin validation in parallel with the elicitation of stakeholder needs, start verification in parallel with derivation of requirements, start integration as soon as system architecture starts, etc.

Figure 3 shows the original, canonical Vee model [14] of which there are now many variations (many of them are inconsistent with the concepts expressed in the original Vee model). The left-hand side of the Vee depicts the decomposition effort as stakeholder needs being mapped into system design concepts and eventually derived into a system specification. At the bottom of the Vee model, those system elements are designed and built, and give way to the right-hand side of the Vee model, which depicts the synthesis or integration effort as those elements are integrated up the system hierarchy with verification and validation occurring at each step. The symmetry between the left- and right-hand sides of the Vee model highlights the interdependencies between each stage on the left with the corresponding one on the right. For instance, the concept of operations on the left side defines measures of effectiveness and performance that are the basis of the system validation occurring later in the project.

The Vee model is particularly suited for system development projects that require a systematic and structured approach, where:

- systems in question are large,
- involve many building components, which have different levels of maturity and therefore risks, and
- have stringent safety and reliability constraints.

The Vee model, or minor variations of it, is probably the most common mode used in industries such as aerospace, defense, and automotive.

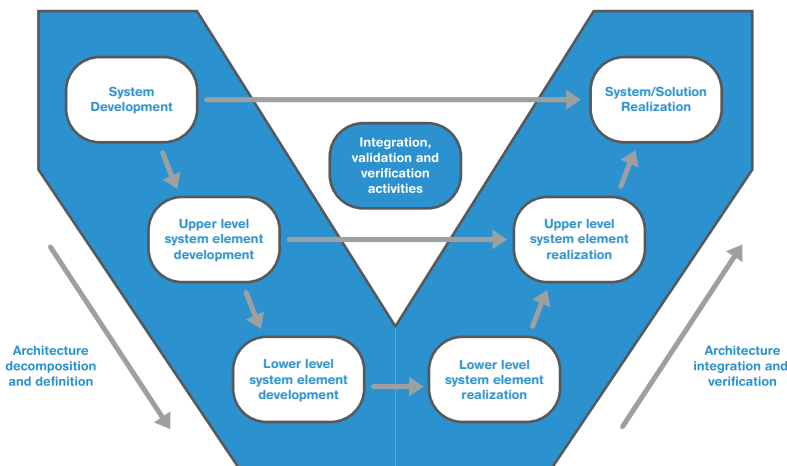


Figure 3. Vee Model

The Vee encounters some of the same shortfalls as experienced with the Waterfall model. The Vee model suffers from poor responsiveness to significant changes in requirements or poor initial assumptions that are not discovered until later in the process. The result is costly rework with the concomitant delays in schedule.

3. EVOLUTIONARY OR AGILE DEVELOPMENT MODELS

Evolutionary or agile models describe a class of highly iterative and incremental approaches to system development. Agile models carry out iteration and incremental development based on short development cycles called *sprints* or iterations. Iteration describes the repetition of the same activities during each sprint such as analysis, design, test, and build for a small portion of the system. We see the incremental aspect through the delivery of value or system capabilities at the end of each sprint. Agile models emphasize the end of a sprint should result in a measurable outcome. For software systems, each sprint usually ends with functional code that can be delivered to stakeholders. For hardware-intense systems, a sprint might not result in actual hardware because of the longer development time required, and instead a sprint may result in risk reduction, an engineering specification, or completion of an engineering analysis. This type of model emphasizes value delivery as an incremental process whether such delivery is some visible portion of the system or advancement of the project at the end of each sprint.

Unlike plan-driven models, the agile models do much less planning and requirements analysis during the beginning of system development. Agile models are highly adaptive and flexible approaches because as the project team plans each new sprint, they will make course corrections based on what they learned during the previous sprint. Hence, agile methods are updating their plans on a regular basis driven by the cadence of the sprints.

The agile models are based on a set of principles informing a mindset and then implemented by multiple, different methods. The principles were described in the Agile Manifesto [2] and are listed in Table 1.

1. Early and continuous delivery of value to the customer
2. Acceptance of changing requirements, important for competitive advantage
3. Frequent incremental delivery of value to stakeholders
4. Users and developers working together
5. Focus on the people, self-organized teams
6. Face-to-face communication is most effective for teamwork
7. Working software is primary measure of progress
8. Development speed should be sustainable indefinitely
9. Continuous attention to good design
10. Keeping the design as simple as possible and avoid doing unnecessary work
11. Best architectures emerge from self-organized teams
12. Regular meetings for teams to reflect on lessons learned

Table 1. Agile Principles

A differentiating aspect of the agile models is that they combine principles on how to develop a system with principles on how to organize the development team. This is a differential aspect with respect to many of the plan-driven models, which provide little guidance about the project management considerations of the system development. The principles dealing with continuous delivery, acceptance of changing requirements, using working software as the measure of progress, steady development speed, constant attention to good design, and simple design are principles all dealing with system development. Agile models uniformly prescribe self-organizing, multidisciplinary teams working in close collaboration with end users (generalizable to general

stakeholders) to deliver value incrementally. Furthermore, they place greater value on face-to-face communication and regular meetings as a means for better coordination and faster recognition and adoption of lessons learned, which are intended to improve the management of development teams.

Agile methods have been widely adopted and routinely practiced in software development projects to improve team collaboration, communication, and flexibility in responding to changes in customer and/or user requirements. The application of agile principles to systems engineering processes, particularly in the development of complex systems with hardware components, remains an area of active research and discussion. Several studies have shown the benefits of applying agile principles to systems engineering processes. Dove et al. [11] studied the application of agile principles in several companies and identified eight principles for agile systems engineering consistent with the agile manifesto including attentive situational awareness, attentive decision making, agile operations concept, product line architecture, shared knowledge management, continual integration and test, common mission teaming, and iterative and incremental development. The International Council on Systems Engineering (INCOSE) has a team that has been looking at how agility can be infused or adopted by systems engineering organizations [38]. Their work is part of the Future of Systems Engineering (FUSE) initiative of INCOSE and emphasizes how development organizations can become more agile through process and workforce development. Therefore, while agile models are proven and the dominant approach in the software, systems engineering is still examining and trying to understand how best to adopt agile principles given the constraints of physicality imposed by hardware.

Next, we present the Spiral model, which was introduced in the 1980s, as an example of an evolutionary model. Then, Scrum is presented because it is widely used in the software industry and the iterative process and concepts from Scrum are heavily borrowed by other evolutionary models. Finally, we discuss two system development models that adopt agile concepts from Scrum for larger projects including both hardware and software: the Scaled Agile Framework (SAFe) and the Disciplined Agile Delivery (DAD).

3.1. Spiral model

The Spiral model was introduced for software development and emphasizes reducing risk through incremental commitment from stakeholders throughout system development [3]. A project following the Spiral model does iterations to progressively develop the definition of a software system and eventually deliver it to a customer. The cycles consist of the four activities (ref. Figure 4):

- 1) determining objectives, alternatives, and constraints,
- 2) evaluating the alternatives, identifying and resolving risks,
- 3) developing and verifying the product, and
- 4) planning the next phase.

Each iteration or spiral results in a prototype or build of the system, albeit often a partial build. Consequently, the project develops and deploys the system in increments.

The Spiral model is suitable in system development contexts characterized by uncertain or evolving requirements and projects where risks are significant and need to be continuously assessed and mitigated throughout the project lifecycle. However, the use of the Spiral model is generally only possible when the system can be deployed in increments. Software systems was one of the original domains for which

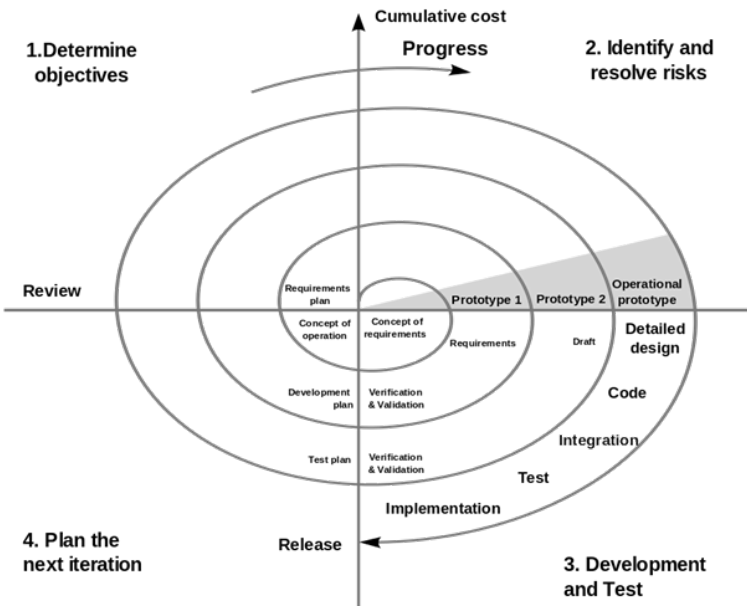


Figure 4. Spiral Model

the Spiral model was intended [4], but it has also been applied to other domains. For example, the ship design process is usually shown as a Spiral model because ship design involves multiple technical specialties with a high degree of interdependence between them. In the ship design spiral, the designers make decisions on hull form, hull size, power, hydrostatics, and so on and must iterate through these decisions on they converge on an acceptable design [13,15]. The spiral model has also been applied to the development of the Global Hawk unmanned aircraft, which was able to deploy capabilities incrementally with each spiral [17].

3.2. Scrum

Scrum is an agile software development model for managing self-organized teams in short iterations called sprints delivering working code at the end of each sprint [34]. The popularity and success of Scrum for software development has motivated people to adapt the Scrum development model to systems involving both hardware and software [e.g., 3, 10]. Figure 5 shows the Scrum process. Teams work from a backlog of work items that were identified in conjunction with a Product Owner representing the voice of the customer. The work items are requirements for features or functions often expressed as user stories. During the sprint planning session, the team selects work items to analyze, design, develop, and test in the next sprint. The sprint is the actual development iteration and is

typically of two weeks duration for software development but may be longer and even vary for hardware development. The team will have a regular meeting to discuss progress of the sprint. A sprint retrospective meeting is held at the end of the sprint for the team to discuss ways in which they can improve. Scrum is a learning process and focuses on the people and interactions between people in developing software.

Scrum works well in dynamic environments where requirements change (because new and/or changed requirements can be worked into the backlog and be addressed in future sprints), as well as with systems that can be built and deployed incrementally. Adoption of the Scrum development model requires implementing a corresponding organizational culture, as with other agile models, because it intertwines development processes with team management and communication principles. An organization that does not subscribe to such working principles as established by the team would likely not be successful in performing a Scrum process.

Because Scrum was originally applied to small projects, there were some initial doubts about its suitability to guide large projects. However, variants such as Large-Scale Scrum (LeSS) [24] were proposed to address this potential shortcoming. While Scrum has been adapted and applied for systems development, it is not widely practiced, and many questions remain as to its applicability in different system development contexts.

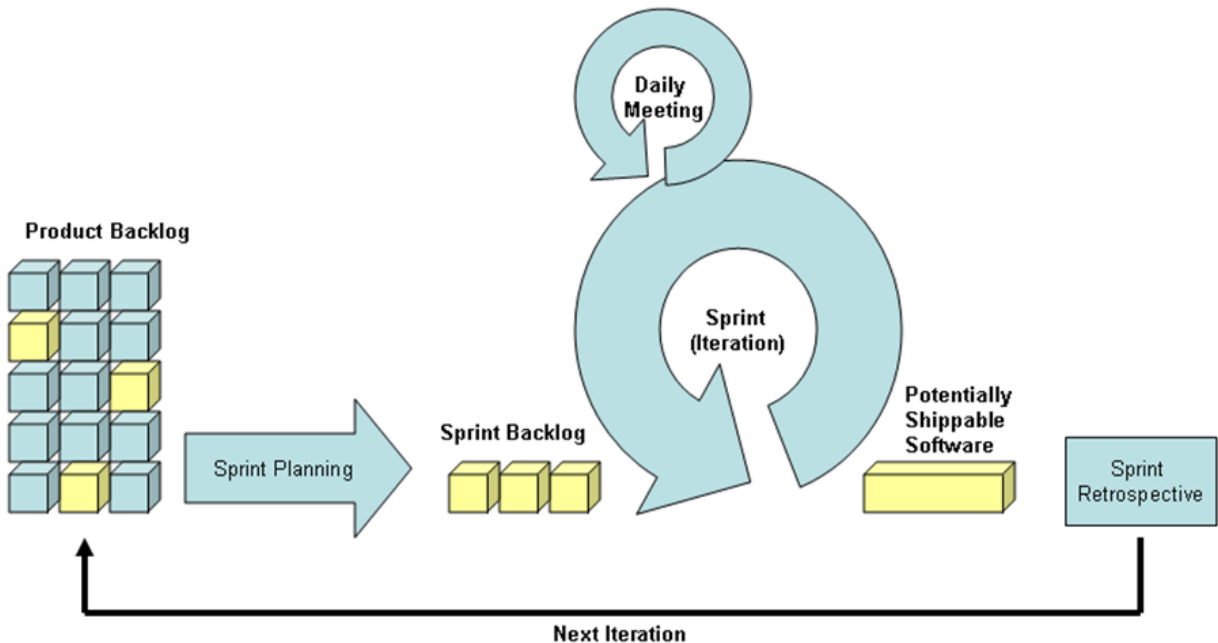


Figure 5. Scrum Process

3.3. Scaled Agile Framework

The Scaled Agile Framework (SAFe) describes a structured and scalable approach to implementing agile principles and practices in organizations with multiple teams working on complex projects [25]. The teams consist of individuals who have the ability to do the analysis, design, build, and test of their work. They work on the systems development through Agile Release Trains (ARTs), which are defined time frames (typically 8-12 weeks) during which multiple teams synchronize their work to develop and deliver a capability of value to the stakeholders. Hence, ART cycles are strictly time-boxed and their scope is allowed to vary.

The teams plan the development goals of an ART through an event called Program Increment (PI) Planning. The PI Planning brings together all teams within an ART to plan and align their work for the upcoming program increment. During PI Planning, teams collaborate to define objectives, break down work, estimate effort, and establish dependencies. SAFe includes a lot of guidance on management of the development team and how it should be organized.

SAFe was developed to address the criticism that agile methods such as Scrum, do not scale to large and more complex system development projects. SAFe does this through the guidance provided for scaling agile from the team level to the program and portfolio levels. It offers different configurations, allowing organizations to tailor their implementation based on their specific needs. In providing the additional guidance on how to structure the development, SAFe is sometimes criticized as sacrificing the agile principles for the sake of greater discipline [31].

3.4. Disciplined Agile Delivery

Disciplined Agile Delivery (DAD) is an agile method that builds on other methods such as Scrum but is intended for the entire life cycle, including operations, for example, unlike other agile methods that focus primarily on development [1]. DAD defines three phases: inception, construction, and transition. During inception, the project identifies the project vision, stakeholders, and initial requirements. In construction, the project develops the system solution on an incremental basis. In transition, the project puts the design through production and engages stakeholders for validation. The DAD applies an iterative and incremental development approach, characteristic of agile models, during the construction phase.

DAD is mainly intended for larger projects by being enterprise aware, meaning DAD recognizes the enterprise context and takes into consideration governance, compliance, and organizational standards. DAD emphasizes the importance of validating the architecture in the earlier sprints and consequently reducing risk. The DAD is particularly appropriate to software-intensive systems in which the development team can establish and react to feedback loops between operations and development (i.e., DevOps, which will be described later).

3.5. Agile methods with hardware

Agile methods, originally developed for software development, may encounter certain difficulties when applied to hardware development due to the inherent physicality and manufacturing aspects involved. We group these constraints here because they apply to all of the agile models previously discussed. Some of the constraints that can limit the application of agile methods for hardware development are:

- **Longer Lead Times:** Hardware often requires longer lead times for designing, ordering materials, and building components compared to software development. This can make it challenging to adhere to the short iteration cycles typically associated with agile methods.
- **Manufacturing Complexity:** Manufacturing hardware involves intricate processes, specialized tools, and strict quality control measures. Unlike software, which can be *easily* modified and updated, modifying and updating physical hardware components often require different tools, facilities, specialized personnel, materials, and so forth. This can limit the flexibility and ability to adapt quickly, which is a key aspect of agile methods.
- **Costly Iterations:** Iterating on hardware designs can be expensive, especially when it involves tooling, materials, and production processes. Unlike software, where changes can be made relatively *easily* and at a low cost, hardware iterations often require additional investments. This can make frequent iterations and experimentation challenging from a cost and resource perspective.
- **Physical Prototyping:** Physical hardware typically requires the creation of prototypes for testing and validation. Building physical prototypes can be time-consuming and costly, which can impact the ability to iterate rapidly and embrace quick feedback loops, as typically done in agile methods.

- **Supply Chain Dependencies:** Hardware development often relies on complex supply chains, involving multiple vendors and lead times for procuring components and materials. This introduces additional dependencies and challenges in managing schedules, coordination, and ensuring timely availability of required resources.
- **Regulatory Compliance:** Hardware products often need to comply with industry-specific regulations and standards. Ensuring compliance can involve extensive testing, certification processes, and documentation. This can add complexity and time to the development process, potentially impacting the agility of the iterative cycles.
- **Law of Physics:** The functioning and performance of hardware systems are bounded by the laws of physics. As a result, strong coupling generally exists between design variables and across engineering disciplines. This makes it very difficult to allocate work in independent tasks.

Despite these constraints, agile principles and practices can still be applied to certain aspects of hardware development. For example, while sprints may not be as short as with software, the principle of frequent scoping and assessment of work can be helpful in keeping a healthy level of productivity and quickly react to misalignments between needs and solutions. This has been shown in practice. For example, Yang et al. [39] showed that the use of agile principles in the development of a hardware product led to improved communication, reduced project risk, and increased customer satisfaction. Similarly, Thakurta et al. [35] showed that the use of these principles in the development of an embedded system led to improved team collaboration, faster development cycles, and reduced project risk. Paasivaara and Lassenius [29] describe Ericsson's long journey of adopting agile to the design and development of their products.

Overall, while the physical constraints of hardware development pose challenges for the direct application of agile methods, careful adaptation, and a tailored approach can help leverage agile principles to enhance collaboration, flexibility, and customer satisfaction in in this type of projects.

4. COMPARISON BETWEEN PLAN-DRIVEN VERSUS AGILE DEVELOPMENT

This section compares the category of plan-driven models versus the category of agile models. We first consider how the two categories of models address the triple constraint of project budget, schedule, and scope.

Plan-driven development models tend to assume a fixed scope-budget-schedule triad (and are often implemented by fixing the scope only), and only act upon them reactively when issues are encountered, such as due to the uncertainty of requirements or the unfolding of technical risks. On the contrary, scope, and to a lesser extent budget are intentionally in flux when using *agile models*. This happens as a result of organizing the work in iterative time-boxed sprints, as explained earlier, so that the project can adapt its tasks and targets as deemed necessary.

It should be noted that, while agile models are generally implemented to fix budget and schedule (letting scope change), and plan-driven methods are generally implemented to fix scope (letting cost and schedule change), these are not constraints imposed by either of the development methods. In fact, either kind of development model can implement techniques that target the fixing of a specific dimension (e.g., Cost as an Independent Variable (CAIV) fixes cost at the expense of schedule and scope [6]). The key difference is whether *changes* are made reactively to problems or intentionally embedded in the development process.

Table 2 provides a brief comparison between how plan-driven models and agile models typologies address core aspects of the business environment, the system to be developed, and the development organization in which they are applied. The table is not intended to be exhaustive, but to broadly hint at the main differences between the two classes of models. As an example, one consideration is the type of business environment the organization and system will operate in. Organizations working in regulated environments and/or dealing with safety critical systems will generally need more planning, documentation, and traceability of requirements. Plan-driven models address these concerns directly. However, this does not mean agile models cannot be used in these environments. For example, Hanssen et al. [16] propose a variation of Scrum, called SafeScrum, demonstrating agile principles in the development of safety-critical systems such as avionics despite the strong belief that agile is inconsistent with the need to comply with stringent safety regulations.

Contingent Factors		Plan-Driven Models	Agile Models
Business Environment	Stability of market and certainty of requirements	Better for stable and unchanging requirements	Better in dynamic environments with changing requirements
	Clarity and certainty of requirements	Better when requirements are clear and known	Better able to deal with ambiguous and unclear requirements
	Regulatory or safety critical environment	Better for complying with strict regulations and policies; ensure traceability with documentation and plan	--
System	Part of a system of system with lots of Interconnections with other systems	Better for planning and controlling those interfaces	--
	Long lead-time items or must plan for scarce resources (e.g., test range)	Better for planning the acquisition of such items and aligning budget, schedule, and resources.	Greater risk of not having such items on hand when needed or available due to limited up-front planning
	Many needed quality attributes (-ilities such as reliability, maintainability, cybersecurity, etc.)	Plan-based models identify and define these requirements early on, which is useful because the quality attributes are often met through multiple aspects of the overall system design	If quality attributes are identified during sprints (especially later sprints), then risk of rework to have the system comply with those requirements because multiple aspects of system might have to change
	Technological risk present	Attempt to develop plan to mature the technology prior to insertion in the system	Rapid learning and risk reduction through iterations
Organization	Highly geographically and organizationally distributed team	The planning and discipline of these models are able to accommodate these scenarios well	Some agile models are less able to handle such organizational structures without significant modification to principles (e.g., difficulty of face-to-face communication). Better for smaller co-located teams

Table 2. Comparison Plan-driven and Agile Models

5. TAILORING DEVELOPMENT MODELS

It is convenient to keep in mind that there is not an ideal or *default* preferred model for a system development. We view the best system development model as one that best fits:

- the organization, its people, and its culture;
- the system to be developed, its complexity, its connectedness to other systems, and the extent of new technologies that it uses; and
- the business environment in which it is developed, the dynamism of the business, and the degree of uncertainty surrounding the development.

For this reason, *tailoring* of the development models should be often called for. Tailoring can take the form of establishing a canonical development model and adapting it to the particulars of the project, it may involve a hybrid approach blending aspects from multiple different development models, or it may be approached by using different development models for different parts of the system (e.g., adopting a Vee model for the hardware components and an agile model for the software). Tailoring relies upon the systems engineering team having deep knowledge of the advantages and drawbacks of each development model and understanding the factors affecting the development, as mentioned earlier (e.g., business environment, system, and organization).

The agile approaches have been demonstrated as being very successful in the software industry, yet there are limitations as aforementioned in applying them to hardware. For this reason, there is interest in hybrid approaches that blend agile and traditional plan-driven development models to strike a balance between flexibility and the need for rigorous development processes. When tailoring, it is essential to address two questions [33]:

- 1) To what degree is agility demanded by the market, technology, and other environmental factors?.
- 2) To what degree can the organization be agile?.

In essence, it is important to coordinate the demands or needs that the system must satisfy with the ability and/or feasibility of the organization to adopt a particular development model [5]. Tailoring, in this sense, should not be seen as something to be done when *ideal* models cannot be used. Instead, tailoring should be conceived as a required step when devising what type of model to adopt. Furthermore, *ideal* models should serve as paradigms. In reality, a development model will generally need to be tailored.

Figure 6 refines Table 2 by showing plan-driven models and agile models not as a dichotomy, but as continua of adequacy between the two for different variables.

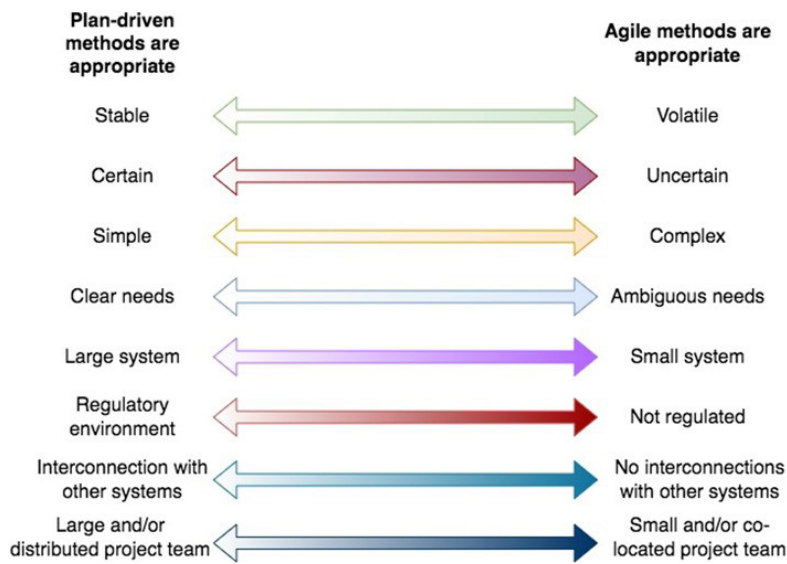


Figure 6. Factors for deciding between plan-driven and agile models

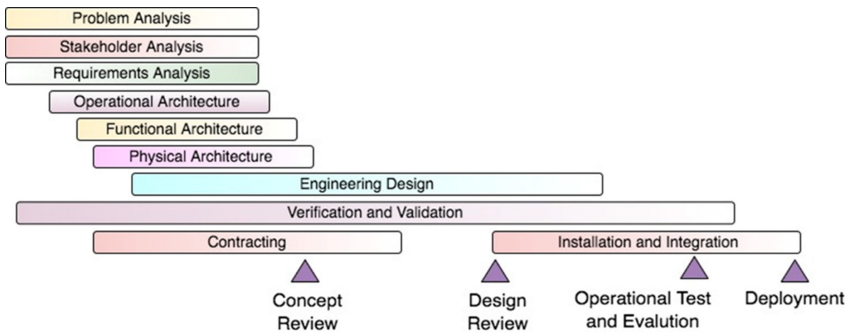


Figure 7. High level plan

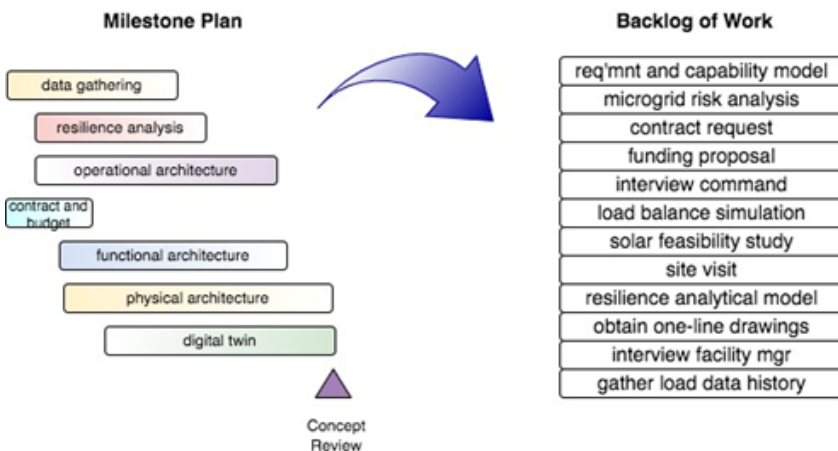


Figure 8. Milestone plan and backlog

One means to combine plan-driven models with agile principles is to adopt and adapt important agile concepts at multiple levels of development. A hybrid method can iteratively and incrementally evolve the system according to agile principles while maintaining some of the predictability available from plan-based models.

As an example, consider hardware-based or hardware-intensive systems. This type of system requires some planning because hardware often requires parts with long lead-times, it cannot be refactored, and customers want to know when the system will be first deployed. Additionally, larger systems often have many interfaces and interactions with other systems that must be planned for and controlled. Interactions in this context also include performance dependencies between the different components. These issues are addressed by macro-planning of the overall development process and by using a *top-down* approach starting with an initial system architecture. Figure 8 shows a high-level view of the system development activities and Figure 8 shows the more detailed planning prior to the next milestone. Both figures show there is extensive parallelism of the activities, but they do not depict how the intensity of effort changes with each activity. For instance, the problem analysis activity might occur through to the concept review milestone, but the amount of manpower and effort will be greater earlier in the project compared to just before that milestone.

Figure 7 shows verification and validation (e.g., testing) occurs *continuously* throughout the process. The frequent testing enables permanent design maturation and risk reduction. Within each phase, there are iterations of understanding, designing, building, and testing ideas through the use of models. Additionally, there are feedback loops and iterations between phases. For instance, as capabilities are analyzed and defined, the team might rethink how they framed the problem and revise their associated analysis. As a result, this process progressively analyzes, designs, and evaluates the stakeholder needs, requirements, and mission to build the architectural products.

6. MERGING DEVELOPMENT AND OPERATIONS WITH DEVSECOPS

System development models have been traditionally designed to end when the system transitions to the utilization phase of its life cycle. While systems may certainly cycle back to development from utilization, development models tend to assume that, in such a case, a new project would be specified. The sharp boundary between development and utilization (also referred to as operations) is being challenged by a novel development model called DevOps.

DevOps emerged from the software engineering community and aims to break down the barriers often found between *system developers* (Dev) and the *system operators* (Ops) [22]. DevOps implements a continuous cycle of developing software, testing it, and pushing the software out to operations who use the software, monitor its performance, and provide feedback to development. DevOps is based on having open and close communication between developers and operators, continuous feedback, continuous integration, and steady operational flow.

DevOps has been and continues to be almost exclusively applied to software-intensive systems. The challenges to adopting DevOps to hardware are evident. Unlike software, new features or functions cannot be pushed out over a network, and moreover, changes to a hardware's performance usually entails changing the hardware itself. DevOps has been challenging to implement even with software for embedded systems [27]. Only recently, do we see DevOps being applied in system domains involving both hardware and software, but with the continuous integration and verification occurring only with the software components [40].

In military systems, the need for security is paramount, and DevOps has morphed to include security considerations, creating a new model concept known as DevSecOps [28]. This approach involves integrating security practices into every stage of the software development process, from design and coding to testing and deployment, to ensure that security is built into the system from the outset. By adopting a DevSecOps approach, military organizations can help to mitigate the risks of cyberattacks, protect sensitive data and systems, and ensure that military operations are not disrupted by software vulnerabilities or failures. Overall, the DevSecOps methodology has emerged as a key approach for developing secure, reliable, and scalable software systems, and its importance is likely to continue growing in military and other high-stakes environments.

7. CONCLUSIONS

The chapter has reviewed the need for, and role system development models fulfill in systems engineering. The life cycle of systems has been described as consisting of six stages of concept, development, production, utilization, support, and retirement. The systems development process covers primarily the stages of concept, development, and production – although recent approaches seek greater integration between development and utilization (or operation). Development models can be broadly categorized as either plan-driven or agile. The plan-driven models are the more traditional systems engineering approaches and include the Waterfall and Vee models.

The Waterfall and Vee models predominantly move through the system development activities in a sequential fashion. Projects following these models start with planning and generally understanding all the system requirements prior to moving onto design activities. However, it has been long recognized that actual implementation of these models involves extensive iterations and recursion of activities.

Agile models have emerged primarily from the software engineering community. Software and its development differ in several important ways from hardware, which helps explain why agile methods become popular there first. Among these differences is that software is intangible and can be deployed over a network. Consequently, software is easy and relatively inexpensive to modify, even after the software is deployed to users. Moreover, the market for software is highly dynamic with changing needs and frequently introducing new technologies. The agile methods exploit the characteristics of software making it easy to change in order to thrive in such a dynamic business environment. Agile models are founded on principles of iterative and incremental development, continuous verification and validation, self-organized teams, continuous integration, and close interaction with users. The Spiral model was one of several other models that started to more formally and explicitly describe the systems development process as being iterative and incremental. Other novel agile development models emerged later, such as Scrum, SAgile, and DAD, which implement these principles.

The chapter has also provided a comparison between plan-driven and agile models along different dimensions to highlight their strengths and weaknesses. In general, plan-driven approaches are suitable for development projects where requirements are knowable, technology is mostly mature, and the environment is stable. They are also useful when audit trails and documentation are necessary such as for safety-critical systems or many defense systems. Agile models, on the contrary, are suitable in environments where there is high uncertainty and dynamism. The chapter observes that many projects might have characteristics suitable for plan-driven and other characteristics for agile models. In such cases, the chapter suggests a hybrid plan-driven and agile model attempting to combine the strengths of each process model. Such tailoring of process models is very important in systems engineering because there is no single model suitable for every project. Instead, informed and knowledgeable systems engineers should tailor or customize the system development models to fit the needs of the organization and the development project.

The chapter described emerging development trends affecting most system development projects such as DevOps. The DevOps concept interleaves development with operations using continuous feedback for the evolution of the system throughout its service life. DevOps as well as other emerging development trends can be incorporated into any of the system development models, and indeed the development models may need to be modified to better exploit these new capabilities.

REFERENCES

1. Ambler, S. W., & Lines, M. (2012). *Disciplined agile delivery: A practitioner's guide to agile software delivery in the enterprise*. IBM press.
2. Beck, K., Beedle, M., Van Bennekum, A., Cockburn, A., Cunningham, W., Fowler, M., ... & Thomas, D. (2001). *The agile manifesto*.
3. Boehm, B. W. (1988). A spiral model of software development and enhancement. *Computer*, 21(5), 61-72.
4. B. Boehm, *Spiral Development: Experience, Principles, and Refinements*, CMU/SEI-2000-SR-008, July 2000.
5. Boehm, B., and Turner, R. (2004). *Balancing agility and discipline: A guide for the perplexed*. Addison-Wesley Professional.
6. Brady, J. (2001). Systems engineering and cost as an independent variable. *Systems engineering*, 4(4), 233-241.
7. Chrissis, M. B., Konrad, M., & Shrum, S. (2011). *CMMI for development: guidelines for process integration and product improvement*. Pearson Education.
8. De Weck, O. L., Roos, D., & Magee, C. L. (2011). *Engineering systems: Meeting human needs in a complex technological world*. Mit Press.
9. Dingsøyr, T., Falessi, D. and Power, K. (2019). "Agile development at scale: the next frontier". In: *IEEE software* 36.2 (2019), pages 30–38.
10. Douglass, B. P. (2015). *Agile systems engineering*. Morgan Kaufmann.
11. Dove, R., Lunney, K., Orosz, M., & Yokell, M. (2023, July). *Agile Systems Engineering–Eight Core Aspects*. In *INCOSE International Symposium* (Vol. 33, No. 1, pp. 823-837).
12. Elssamadisy, A., Abu-Tayeh, G., & Brown, T. (2018). *Scaling agile and lean development in the enterprise*. Pearson Education.
13. Evans, J. H. (1959). Basic design concepts. *Journal of the American Society for Naval Engineers*, 71(4), 671-678.
14. Forsberg and Mooz, 1998 à Forsberg, K. and Mooz, H. (1998), 7.17. *System Engineering for Faster, Cheaper, Better*. *INCOSE International Symposium*, 8: 917-927. <https://doi.org/10.1002/j.2334-5837.1998.tb00130.x>.
15. Gale, P.A., "The ship design process," in *Ship Des. Construction*, vol. 1, T. Lamb, Ed. Alexandria, VA: SNAME, 2003, ch. 5.
16. Hanssen, K. G., Stålhane, T., & Myklebust, T. (2018). *SafeScrum®—agile development of safety-critical software*. Springer Nature Switzerland AG.

17. Henning, W.A., and Walter, D.T., (2005). Spiral development in action: a case study of spiral development in the Global Hawk Unmanned Aerial Vehicle program. Master's Thesis. Naval Postgraduate School, Monterey, CA.
18. Jim Highsmith and Alistair Cockburn. "Agile Software Development: The Business of Innovation". In: Computer 34.9 (2001), pages 120–127.
19. Ibarra H., and Scoular, A. (2019). "The Leader as Coach". In: Harvard Business Review (November - December 2019).
20. INCOSE (2023). SE Vision 2035: Engineering Solutions for a Better World. International Council on SE. Retrieved September 1, 2023 from <https://www.incose.org/about-systems-engineering/se-vision-2035>.
21. ISO/IEC/IEEE International Standard - Systems and software engineering -- System life cycle processes, 2015-05-15.
22. Kim, G., Humble, J., Debois, P., & Willis, J. (2016). The DevOps handbook: How to create world-class agility, reliability (2nd ed.). IT Revolution Press. <https://itrevolution.com/book/the-devops-handbook/>.
23. Knaster R. and Leffingwell, D. (2017). SAFe 4.0 distilled: applying the Scaled Agile Framework for lean software and systems engineering. Addison-Wesley Professional.
24. Larman, C., & Vodde, B. (2016). Large-scale scrum: More with LeSS. Addison-Wesley Professional.
25. Leffingwell, D.: Scaling Software Agility: Best Practices for Large Enterprises. Pearson Education, Boston (2007)
26. Lockey, S., Gillespie, N., Holm, D., & Someh, I. A. (2021). A review of trust in artificial intelligence: Challenges, vulnerabilities and future directions. Proceedings of the 54th Hawaii International Conference on System Sciences.
27. Lwakatare, L. E., Karvonen, T., Sauvola, T., Kuvaja, P., Olsson, H. H., Bosch, J., & Oivo, M. (2016, January). Towards DevOps in the embedded systems domain: Why is it so hard? In 2016 49th Hawaii international conference on system sciences (hicc), pp. 5437-5446.
28. Miller, A. W., Giachetti, R. E., & Van Bossuyt, D. L. (2022). Challenges of Adopting DevOps for the Combat Systems Development Environment, Defense Acquisition Research Journal, 29(1).
29. Paasivaara M. and Lassenius, C. (2019). "Empower your agile organization: Community- based decision making in large-scale agile development at Ericsson". In: IEEE Software 36.2 (2019), pages 64–69.
30. Pahl G. and Beitz, W. (2013). Pahl/Beitz Konstruktionslehre: Grundlagen erfolgreicher Produktentwicklung. Methoden und Anwendung. Springer-Verlag, 2013.
31. Putta, A., Paasivaara, M., & Lassenius, C. (2018). Benefits and challenges of adopting the scaled agile framework (SAFe): preliminary results from a multivocal literature review. In Product-Focused Software Process Improvement: 19th International Conference, PROFES 2018, Wolfsburg, Germany, November 28–30, 2018, Proceedings 19 (pp. 334-351). Springer International Publishing.
32. Scheithauer and Forsberg, 2013. à Scheithauer, D. and Forsberg, K. (2013), 4.5.3 V-Model Views. INCOSE International Symposium, 23: 502-516. <https://doi.org/10.1002/j.2334-5837.2013.tb03035.x>.
33. Stelzmann, E. (2012). Contextualizing agile systems engineering. IEEE Aerospace and Electronic Systems Magazine, 27(5), 17-22.
34. Schwaber 1997 à Schwaber, K. (1997). SCRUM Development Process. In: Sutherland, J., Casanave, C., Miller, J., Patel, P., Hollowell, G. (eds) Business Object Design and Implementation. Springer, London. https://doi.org/10.1007/978-1-4471-0947-1_11.
35. Thakurta, R., Mukhopadhyay, D., & Das, D. (2020). Applying agile practices in embedded system development: An industrial case study. Journal of Systems and Software, 168, 110660.
36. Ullman, D. G. (2019). Scrum for hardware design: supporting material for the mechanical design process.
37. Wade A Henning and Daniel T Walter. Spiral development in action: a case study of spiral development in the Global Hawk Unmanned Aerial Vehicle program. Technical report. Naval Postgraduate School, Monterey, CA, 2005.
38. Willett, K. D., Dove, R., Chudnow, A., Eckman, R., Rosser, L., Stevens, J. S., ... and Yokell, M. (2021, July). Agility in the Future of Systems Engineering (FuSE)-A Roadmap of Foundational Concepts. In INCOSE International Symposium (Vol. 31, No. 1, pp. 158-174).
39. Yang, J., Lappas, T., Egan, M., & Bagaria, N. (2019). Applying Agile methods to hardware product development. Journal of Systems and Software, 154, 1-14.
40. Zaeske, W., & Durak, U. (2022). DevOps for Airborne Software: Exploring Modern Approaches. Springer Nature.

BIOGRAPHIES

DR. RONALD GIACHETTI

Dr. Ronald Giachetti is a Professor of Systems Engineering at the Naval Postgraduate School (NPS) in Monterey, CA. He teaches and conducts research in model-based systems engineering, system architecting, and system of systems engineering. He has 30 years of engineering experience in academia and industry. He has published



extensively with over 50 technical papers including a textbook on the design of enterprise systems. He has lectured internationally in Europe, Latin America, and Asia. He has held multiple leadership positions including Chair of the Systems Engineering Department, Dean of the Graduate School of Engineering at NPS, and Chair of the Corporate Advisory Board (CAB) for the International Council on Systems Engineering (INCOSE). He holds engineering degrees from Rensselaer Polytechnic Institute, New York University – Polytechnic, and North Carolina State University. Outside of engineering, he is an avid sailor and races in local and national regattas.

JUAN CARLOS LARIO MONJE

Juan Carlos Lario Monje is an aeronautical engineer with a multidisciplinary experience in several engineering fields such as safety, quality, programs, offsets, contracts, definition & handling of requirements, integration, V&V of systems and Certification & Qualification. He has acquired a global perspective on aeronautical international



programs (civil/military), since its development phase to later serial production and final entry into service, with specific involvement on aerodynamics, Electronic Warfare and Electro-Optical systems. With direct participation in the main European aeronautical programs, both civil (A380, A350) & military (A400M, EF2000, Tiger-HAD, NH90), he has reached a profound knowledge of Spanish aeronautical Industry core businesses (Airbus, Indra), defence programs end-users (SP Army, Air Force) and European Contract agencies (NETMA, OCCAR). With an endless curiosity, he is an enthusiastic professional of continuous learning and teaching. CSEP certified by INCOSE.



"All models are wrong, some are useful."

G.E.P. Box.

Model-Based Systems Engineering

David Long, *Blue Holon* (david.long@incose.net)

Dr. Kaitlin Henderson, *Radiance Technologies* (kaitlin.henderson@radiancetech.com)

Belinda Misiego, *Isdefe* (bmisiego@isdefe.es)

Abstract

This chapter presents Model-Based Systems Engineering (MBSE) as an evolution of the discipline that leverages the power of computer models. It identifies the main elements that MBSE should address to be successful and demystifies some common misconceptions that lead to weak and/or poor practices in organizations. After presenting some unprecedented capabilities by adopting MBSE approaches, the chapter provides some guidance to facilitate the successful adoption of MBSE.

Main topics include the effects of formalizing systems engineering, divergence and convergence of semantics, and authoring, review, and control of configuration in model-based environments.

Keywords

Model-Based Systems Engineering (MBSE), ontology, metamodel, Systems Modeling Language (SysML).



1. INTRODUCTION

Model-Based Systems Engineering (MBSE), Model-Based Engineering (MBE), Digital Thread and Digital Twin (DT), Digital Engineering (DE), and Digital Transformation (yet another DT). There is an explosion of concepts and the corresponding acronym soup as the power of digital is applied to systems engineering and the greater engineering lifecycle. This chapter will try to clarify the differences between the various concepts and how they interrelate. It will move beyond the marketing, myth, and misconception to a practical understanding of what digital transformation means for systems engineering, the fundamentals needed to be known, and the expected value to be achieved.

Before diving into discussions about what MBSE is, let us look at the environmental context from which it has emerged. In recent decades, the development and capability of products have evolved. First, base products built on electro-mechanical technologies have largely moved to smart products leveraging electronics and software to bring new capabilities and an enhanced user experience. Next, sensors and networks have been introduced to create smart, connected products with capabilities such as knowing exactly when the next train will arrive and how many seats remain available. After, these products needed to be more and more connected until becoming a product system, with all the elements coordinated. Today, traditional and intelligent systems are collaborating as systems of systems (SoS) to meet the needs of society (e.g., airport operations in which the flight information system, the transportation system, the operational safety system, and all the other systems involved collaborate in a coordinated and connected manner).

In the days of electro-mechanical products, the interactions between parts, components, and systems were somewhat limited. Given this low coupling between components and between the design teams creating them, documents represented an appropriate solution for capturing architecture and design data. In a document-based world, data is dispersed throughout a multitude of documents. However, as new technologies are introduced and systems become larger and more complicated, the number and complexity of interfaces increases dramatically. Data dispersed between artifacts could generate inconsistencies between them. Plus, keeping hundreds of documents updated for a complex system can be an arduous task. Design inconsistencies may occur due to interpretation discrepancies of the information from use of approaches, language, or diagrams without common semantics between stakeholders. Discovering

inconsistencies in the design (and therefore significant rework) can lead to potential delays and extra costs in development, and undiscovered inconsistencies can result in system failure [1].

Looking at the rapidly evolving technological progress in the world, it is fair to say that the needs from engineering design processes have exceeded what the capabilities of traditional systems engineering can provide. In fact, in 2014 INCOSE published the “System Engineering Vision 2025” [2], with a statement of where the industry had to move to in order to solve society needs. The publication highlighted a number of challenges, including:

- Mission complexity is growing faster than our ability to manage it.
- System design emerges from pieces, rather than from architecture.
- Knowledge and investment are lost at project life cycle phase boundaries.
- Knowledge and investment are lost between projects.

What worked for electromechanical systems in the 1950s, 1960s, and 1970s is not sufficient to address today's needs and technologies. Systems engineers started to make the leap to model-based approaches to respond with agility and efficiency to the complex and changing world.

In 2021, INCOSE published the “System Engineering Vision 2035” [3]. One of the headlines of this publication is “The future of systems engineering is predominantly Model-Based.” By 2035, it is expected that a family of unified, integrated MBSE-Systems Modeling and Simulation (SMS) frameworks will exist.

But what is MBSE? The challenge is the meaning of MBSE is very muddy, which traces to the breadth and ambiguity of what constitutes a model. One technical definition of model is “A physical, mathematical, or otherwise logical representation of a system, entity, phenomenon, or process” [4]. A broader definition of model is “a graphical, mathematical (symbolic), physical, or verbal representation or simplified version of a concept, phenomenon, relationship, structure, system, or an aspect of the real world” . A computational fluid dynamics representation of air flowing over a wing satisfies these definitions, but so does a drawing of a process flow and a traditional interface control document. Because of the breadth of what constitutes a model, different practitioners will have

very different interpretations of what MBSE is. Most people will cite INCOSE's definition of MBSE, which is "the formalized application of modeling to support the activities of systems requirements, design, analysis, verification and validation starting in the design phase conceptual and continuing in the development stage and subsequent stages of the life cycle," but the ambiguity around "model" and "modeling" remains.

Simply put, MBSE represents a new way of performing systems engineering; one that uses model-based techniques to perform systems engineering tasks instead of traditional document-based ones [5]. It is about moving from capturing data in natural language documents to richer representations that reflect and communicate the phenomenology, architecture, and design of systems, ideally in a machine-readable form. The central aspect is the system model, from which all other artifacts are derived. It serves as the connective tissue that binds together the greater digital enterprise that enables engineering. If MBSE were to emerge today, it would have been most likely referred to it as digital systems engineering.

MBSE should make system descriptive and analytical models explicit, coherent, consistent, and actionable by both humans and computers. It should reflect an evolution from low-fidelity representations in documents to higher-fidelity, richer representations that machines can read and interpret. It should improve granularity of knowledge capture for knowledge management, analysis, and learning. It should enable one descriptive architectural model connecting multiple analytical models, which together represent design with the requisite degree of rigor for the problem and solution at hand.

MBSE should also leverage models for communication and analysis, represent "authoritative data" for system design and specifications, ensure consistent design and specifications, and provide an explicit system model to engineering teams. In short, MBSE should be an evolution, not a revolution, in thinking and approach leveraging modern technologies and capabilities to better represent data, information, and knowledge. While evolutionary, this change offers transformative results. MBSE is expected to improve system quality, reduce costs, shorten development times, integrate new technologies and give digital continuity with manufacturing and operations.

2. NECESSARY (AND IDEAL) ELEMENTS OF MBSE

There is a fundamental difference between models in systems engineering and model-based systems engineering. Engineers have always used models to understand and reason. The output of engineering has always been some type of model that helps analyze and advance the understanding of a problem and solution. MBSE leverages modern techniques to capture and represent the fundamental information required to engineer a system. It has traditionally been more on the descriptive and architectural side, although not inherently limited to them, extending from a first expression of need through the whole system life cycle. These models help elicit, capture, and represent a system so that those computational techniques (analytical models, modeling & simulation, etc.) traditionally applied can continue to be used.

In the world of engineering design, architectural models connect the idea behind a design solution with its implementation as a real system. They are the way in which the current working team is aligned, communicate with a higher awareness to minimize misunderstandings, and capture knowledge over time. These models attempt to represent the entities of the engineering problem and their relationship to each other and connect them to the proposed solution or existing mechanism that addresses the problem. If one can properly characterize functionally (i.e., what the system does) and physically each part of the system and its interfaces, as well as the interactions and exchanges between them, then it becomes possible, at the right degree of fidelity and precision, to pass a detailed component description down to a subject matter expert to finish out the design in parallel. The model used in this way is the centerpiece of MBSE.

Four elements are critical to a model [6]:

- **Language:** The modeling language enables the clear expression and representation of the model, so that understanding and insight can arise. The language must be clear and unambiguous to depict the model accurately and understandably.
- **Structure:** Structure allows the model to capture system behavior by clearly describing the relationship between system's entities.
- **Argumentation:** A model must be capable of making the critical "argument" that the system fulfills the stakeholder's needs. The model must represent the system design in such a way that the design team can demonstrate that the system accomplishes the purposes for which it is designed.

- **Presentation:** A model must include some mechanism to present the argument in a way that can be seen and understood by the users, which may include engineers, customers, and other stakeholders.

These elements can be defined by using metamodels, semantics, and ontologies. Metamodels, semantics, and ontologies build the underlying formalities that make so many of the key benefits of MBSE possible (e.g., collaboration, modeling sharing, reuse, reduction of ambiguity, etc.). A metamodel defines the syntax, constraints, and patterns that make up the modeling language that is used when creating a model. An ontology can be thought of as a type of metamodel that defines a common set of terminology, relationships, and context for a given domain, incorporating precise semantics to the terms and relationships.

Figure 1 shows an example of part of a metamodel for systems engineering. It captures (and constrains) the different concepts that the systems engineer may use to support

their work, aligning the understanding and interpretability that every member of the team has. For example, an engineer could not distinguish between a requirement and a specification, as the metamodel does not offer different concepts for those two terms. At the same time, the metamodel does not allow components to directly exchange inputs and outputs but forces the engineer to think about the functions that the component performs to think about inputs and outputs. The impact of this is not only one of fostering understanding and guaranteeing certain good practices when reasoning about systems engineering information, but it also allows to construct machine-readable models to enable better accessing and processing such information. In this metamodel example, one could see how an engineer could query the underlying model to identify all verification requirements related to a function (through a relational path through requirements, components, and finally functions), or enforce rules to guarantee that every component must execute a function, or every function must have at least one input and one output to be consistent with systems theory.

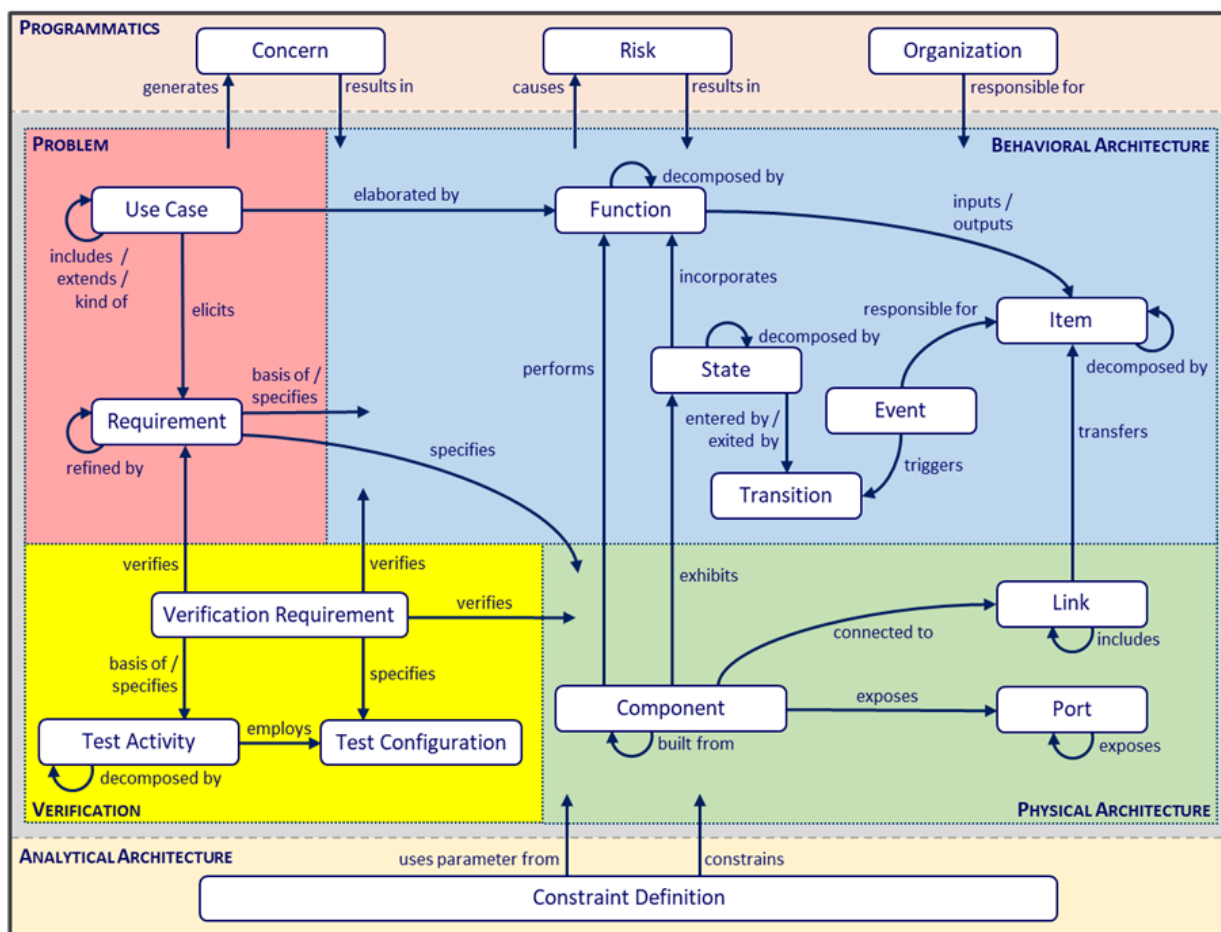


Figure 1. Example of a systems engineering metamodel

Adding precise semantics to a metamodel enables the use of axioms, which afford reasoning capabilities. Whereas a regular metamodel can only guarantee syntactic correctness, an engineer can leverage the semantics of a model to infer aspects of the model that stem from the meaning embedded in the model. For example, whereas a regular metamodel can only check that every Component executes a Function, an ontology affords the possibility of checking that a specific instance of a Function actually represents a function. Imagine that an engineer defines *snake* as a Component and *fly* as a function, and creates the relationship that the *snake* performs the *fly* function. This is syntactically correct, and a regular metamodel would consider it valid. However, an ontology could identify that the model is not sensible because snakes cannot fly.

There is no unique design for metamodels or ontologies; different organizations may prefer different ones to better accommodate their specific context and needs. However, the metamodel and/or ontology should not be arbitrary. Concepts, relationships, semantics, and axioms must be meaningful, internally consistent, and ideally consistent with systems engineering theory and principles, if one would want to benefit from its structure and machine-readability (if implemented as such).

George Box famously said “All models are wrong, but some are useful. The question is how wrong a model can be and still be useful.” Whenever engineers deal with models, they must be deliberate about defining and understanding the purpose behind that model so that the model is fit for purpose in its type, scope, and level of fidelity. MBSE can be valid at any point along the system life cycle, as long as the investment of effort into the model is aligned with the appropriate purpose. For example, if the model is being used in the front end of the lifecycle, then a high-fidelity representation is highly unlikely to be necessary. In place of precision, it may be better to seek elicitation: a better understanding and alignment with the stakeholders to elicit the real needs at the architectural phase. Whether it is fresh design or reengineering, early life cycle, or late life cycle, if the purpose is identified and kept in the forefront, then MBSE can deliver value. If it is not, it is easy to fall into the trap of modeling for the sake of modeling, which defeats the purpose of MBSE. The usefulness of a model is clearly tied to understanding its intended purpose.

The implementation of MBSE requires two pillars in addition to the modeling language, the tool that enables the creation and visualization of the models and the method with which MBSE is implemented [5].

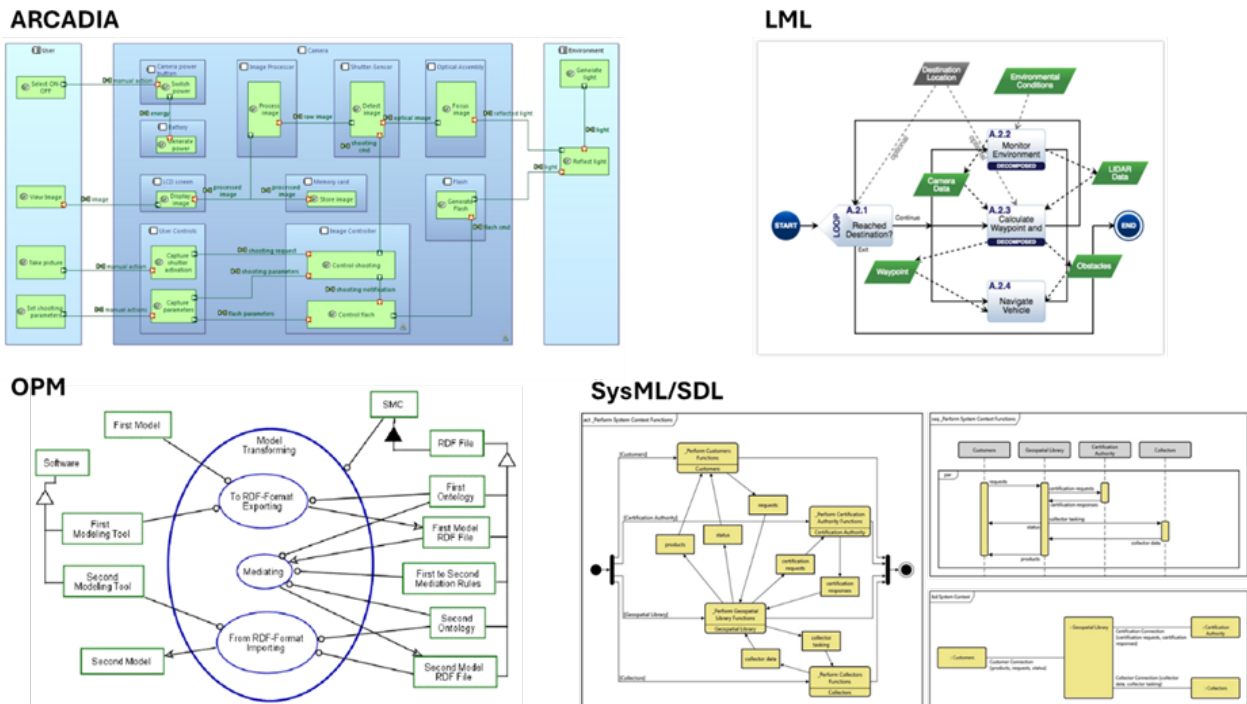


Figure 2. Examples of system models created in different modeling languages

The Systems Modeling Language (SysML) is the dominant language associated with MBSE, but it is important to note that SysML does not equal MBSE. In fact, SysML is not the only language, nor is it always the right language. SysML was developed as a profile of the Unified Modeling Language (UML) developed by the software engineering community to help close the communication gap between system and software engineers during the rise of software-intensive systems. In this sense, SysML is a general-purpose graphical language that is intended to model systems, not necessarily to model all aspects of systems engineering, despite being abused in this sense by the practicing community. Other modeling languages include the Lifecycle Modeling Language (LML), the Object-Process Methodology (OPM), and various languages tied to specific tools such as Capella/Arcadia and Vitech Corporation's Systems Definition Language (SDL). Some graphical representations are shown as examples in Figure 2.

Modeling tools are a special class of tools that are designed and implemented to comply with the rules of one or more modeling languages and enable users to build well-formed models in these languages. Good MBSE tools are far more than diagramming tools. The diagrams are not the model itself; they are merely views of the underlying model which contains a set of elements and relationships that are shown in the diagrams. Much as a computer-aided design (CAD) tool can present top, front, and side views of the geometric model of a part, MBSE tools produce a variety of visualizations of the underlying system model. Some examples of MBSE tools include Cameo Systems Modeler™ (Dassault Systemes), Capella™ (Eclipse open source), Enterprise Architect™ (Sparx Systems), GENESYS™ (Vitech Corporation), Innoslate™ (SPEC Innovations), and Rhapsody™ (IBM). Examples of some of their user interfaces are shown in Figure 3.

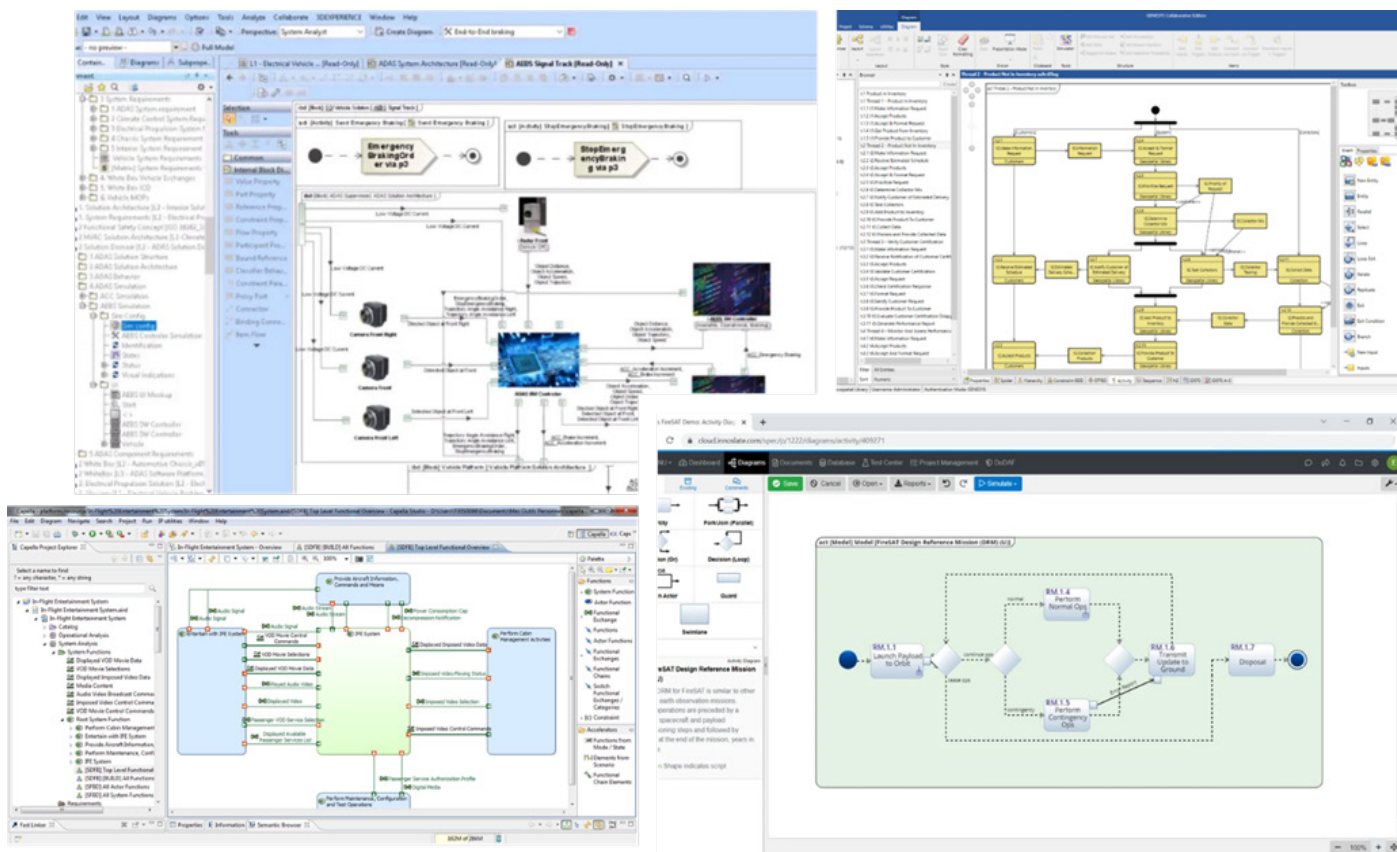


Figure 3. Examples of the user interfaces of different MBSE tools

Modeling methods and/or methodologies can be seen as a roadmap with a set of tasks that ensures the entire team builds the model consistently and works towards a common purpose. A method helps to define the scope; how deep and broad the approach should be towards the model (also based on the opinion of the development teams and the time available). Some general methodologies include INCOSE Object-Oriented Systems Engineering Method (OOSEM), Systems Modeling Toolbox (SYSMOD), Object-Process Methodology (OPM), and the Integrated Systems Engineering and Pipelines of Processes in Object-Oriented Architectures (ISE&PPOOA). It should be noted that these methodologies, while developed with some level of generality in mind, may not fit the needs of every organization. In fact, the adoption of a particular MBSE methodology requires adopting a specific approach to systems engineering. Therefore, ad-hoc methodologies can however be implemented by an organization to better align their implementation of MBSE with their working processes and specific adoption of systems engineering practices. Furthermore, because of this strong connection between MBSE methodologies and systems engineering practices, consistency with systems engineering standards may constraint the implementation of MBSE or vice versa. The adoption of a specific MBSE methodology may require the change and/or tailoring of existing standards.

Finally, people are central to implementing MBSE. When putting together a team to perform MBSE, the team must collectively exhibit expertise in systems engineering, the modeling languages of choice, and the modeling tools of choice, besides any other specific expertise required to complete the project at hand (e.g., different analytical methods, application domain, etc.). Some teams opt to split the expertise between different team members, while some other teams opt to guarantee that all team members have expertise in the three areas. There are advantages and disadvantages to each approach (e.g., splitting expertise accelerates the learning curve but is fragile and less scalable), but explaining them is outside of the scope of this chapter. It is not known where practitioners and professionals will be 15 years from now, but the state of practice and the evolution of MBSE is very fluid and volatile at the moment. Therefore, it seems fair to suggest right now looking for a team that has the characteristics fit for purpose given the business need and context of application.

3. MODELS ARE MORE THAN JUST DRAWINGS

When someone unfamiliar with MBSE looks at an MBSE tool, they will see a collection of drawings and wonder how this is different from using something like Microsoft PowerPoint or Microsoft Visio. The power behind MBSE comes from the underlying data structures, syntax, languages, etc. that make up the actual model (ref. Figure 4). What a diagram shows is considered a representation or view of the system model, as explained earlier. The diagramming capability of MBSE is a great tool to communicate with different stakeholders, but what makes it actually model-based is the connectivity between different components that are unambiguously represented due to the defined syntax and semantics of the modeling language.

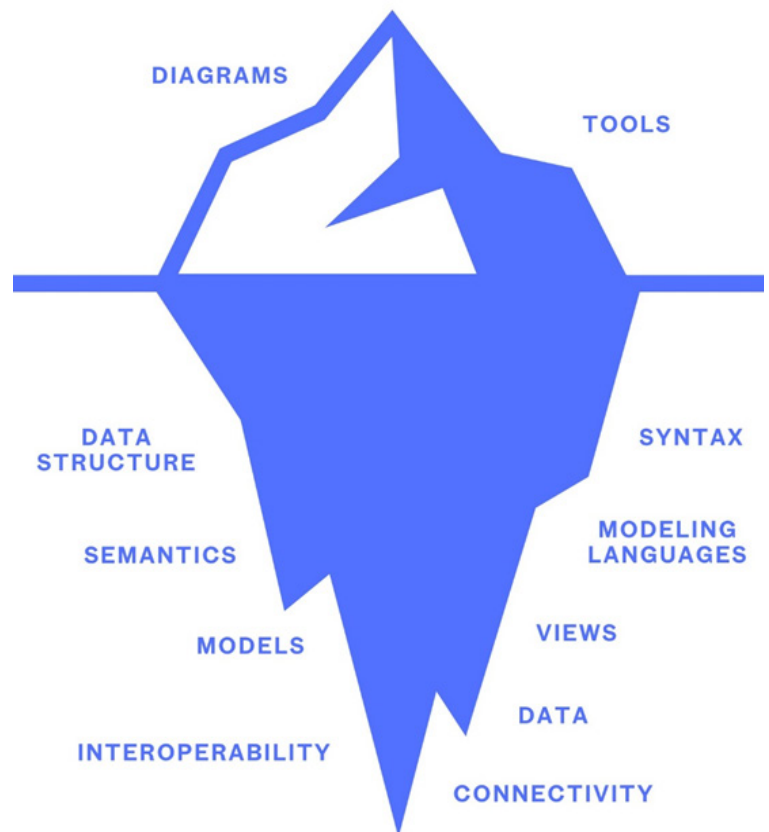


Figure 4. Iceberg model of what a user sees in a diagram versus the information contained in the model

A classic systems engineer will often state that a whiteboard is one of their greatest tools. They can go to the whiteboard, begin a sketch, draw bubbles and clouds and arrows, and label the components. This is useful at a high level and is very good for human-to-human alignment. But then, as the development progresses towards a more detailed design, that flexibility in language and symbology becomes a hindrance because greater precision is needed. An agreed upon meaning of symbols and terminology is necessary to avoid miscommunications and misalignments. There needs to be rigor in the language to ensure effective communication and reasoning by both humans and computers. Flexibility may be useful to understand concepts in the problem space, but if reusing components is desired, and eventually get to connecting models with other tools, there needs to be precision and rigor.

Discussing the precision and consistency of defined terminologies requires coming back to ontologies. Establishing an ontology is fundamentally saying that words, the interrelationships between the words, the concepts that they embody, and the context within which they are valid are defined. The ontology is the underlying knowledge architecture that enables capturing individual pieces of data unambiguously and reflect the interrelationships to represent information and knowledge. However, not all language

models are built upon ontologies and/or the modeling language may hide meaning in its constructs that is unknown to the team members. An example from a research study that explored this differentiation between drawing and model follows.

A number of SysML experts were asked to evaluate the behavior of a system (specifically a car) given a system model and a starting condition [7]. The behavioral model was represented as a SysML state machine diagram, which is shown in Figure 4. Participants were asked to describe the behavior of the car when it was in the *Braking* state and the conditions *releaseBrake* and *speed = 0* occurred simultaneously. They were offered multiple choices to answer: the car will not experience those two conditions simultaneously when in that state, the situation is outside of the scope of the model, the system automatically defaults to one of the transitions, or this is a non-nominal situation that is not captured by the model. The responses were uniformly distributed; in other words, there was no agreement between the experts. The study showed that, while the experts could read the same diagram, their understanding of what the model conveyed was different, and this was due to a lack of understanding of what the modeling constructs embedded in the model beyond the graphical representation of some of its aspects.

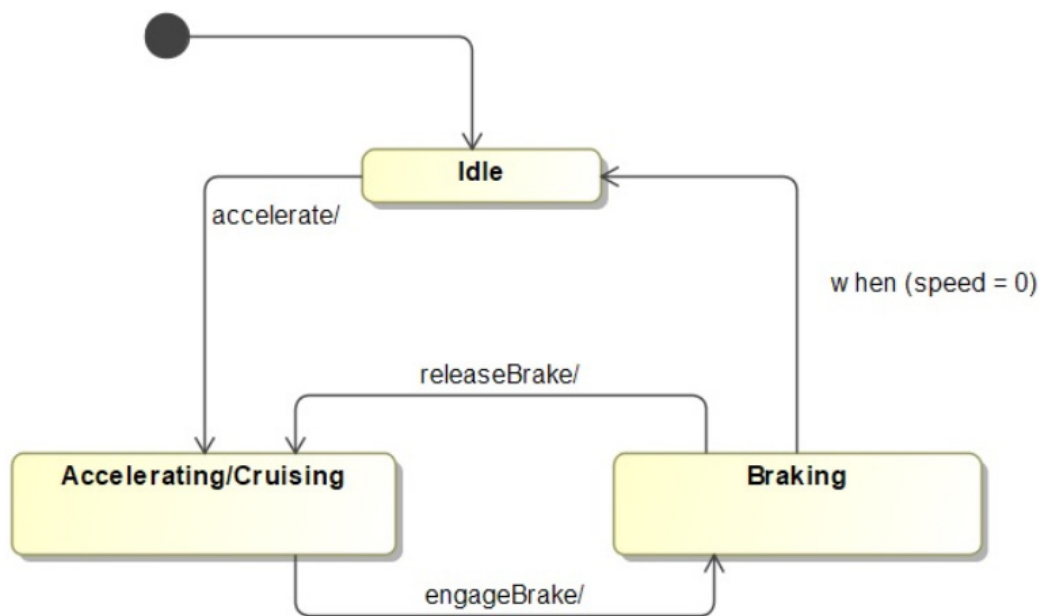


Figure 5. Behavioral model of the car system using a SysML state machine diagram [reproduced from [7]]

One of the challenges of ontologies is that there are multiple ontologies in play at the highest level of the system encapsulation. When the focus is on how systems engineering is performed and characterizing the resulting system of interest, there is a rather small core ontology (a general systems ontology) that enables to capture, communicate, and reason about concepts such as requirements, functions, exchanges, components, interfaces, and interrelationships between those concepts (e.g., satisfying a requirement with a function or allocating a function to a component). As the analytical and the phenomenology necessary to develop the system of interest are brought in, the scope of the required ontology explodes. The phenomenology for an aircraft, for an insulin pump, or for an IT system are very different, therefore, the language around them (the domain specific ontology) is different. The path to success in MBSE may lie in picking a base systems ontology that addresses the scope and language of the team, then specializing it to include key engineering concepts and concepts that align with the organization's methods.

A basic understanding of ontologies better justifies why models are more than just drawings. Returning to the analogy of CAD, the underlying ontology of geometry is points and vectors. If the canopy of a plane is shifted back by 50 centimeters, this implies that the underlying points and vectors in the data model are also being changed. The top and front views presented by CAD would then adjust to show the new position of the canopy. Similarly, in MBSE, if a component is added to the compositional diagram of a system, this is more than adding a box on a drawing. It is being specified that that system has a new subcomponent. Any representation of the system's physical architecture would then reflect this new subcomponent.

4. MBSE IS NOT A SILVER BULLET; GOOD SYSTEMS ENGINEERING IS A PRE-REQUISITE FOR GOOD MBSE

Despite its inclusion in the name, MBSE should not be about modeling. MBSE should be about doing systems engineering while properly leveraging models and a model-based approach. Without a proper understanding of the fundamental principles, processes, and methods for systems engineering, it is possible to spend a great amount of time and money modeling, but it will not be value added. Unfortunately, this is a common mistake that organizations make when adopting MBSE: purchasing several tool licenses and offering their

engineers short training courses on the tool and/or modeling language, without guaranteeing a strong underlying expertise in systems engineering. This is exacerbated by a growth in professional modelers (expertise in modeling is relatively easy to build) at the expense of poor systems engineering practices (gaining expertise in systems engineering is hard).

The key to embracing MBSE is to focus on systems engineering and recognize that “model-based” is largely using computer-aided techniques to better execute what was previously done in a document-based environment. MBSE is not a point of departure for systems engineering, but rather an evolution to keep pace with and leverage the rapidly changing technological landscape. In fact, technology will continue to evolve, enabling engineers to better represent, communicate, and analyze data tomorrow than they can today. What is being today called MBSE is the beginning of a continuing evolutionary journey for systems engineering.

It is also important to recognize that technology is not always a blessing. A common trap of MBSE is that it may entice an engineer to be prematurely precise, particularly when heavily relying on professional modelers with little systems engineering expertise, which comes in at the cost of too much effort and over constraining the design envelope early in the design journey. Good SE practices and engineering judgement must be front of mind when progressing through the system lifecycle moving from higher levels of abstraction to more detailed understanding. The level of accuracy to be achieved should be driven by the purpose of the modeling effort. The same applies to precision. Determining the level of accuracy and precision that is necessary comes back to sound SE principles, engineering judgement, and knowing the purpose behind the modeling effort.

5. NOVEL CAPABILITIES ENABLED BY MBSE

There are numerous claims of benefits MBSE provides across literature sources and from observations made by practitioners. Some of these benefits include better communication, improved consistency, reduced cost, reduced time, reduced errors, and improved system understanding [8]. A key strength of MBSE lies in the ability to clarify communication and shared understanding across the team. It is the ability to better capture information over time and free it from so called drift. Recollections and understanding change, so formally capturing the system provides a more accurate, more precise representation over time. A clarified representation highlights

where there are gaps in thinking or inconsistent understanding across the team. A higher fidelity descriptive architectural representation helps align the team – clarifying understanding, exposing assumptions, and reflecting the design journey behind the current solution. At some point in implementation there will be a question about a design decision, or a new technology, or a requirements change, so capturing the design journey is as critical as the resulting design.

Many of the benefits cited above stem from MBSE establishing an Authoritative Source of Truth (ASOT) for systems [9]. Essentially, this is formalizing that “connective tissue” discussed earlier. In the early days of MBSE, there was a vision of MBSE being established as the single source of truth; a central element that guaranteed all data used in a project were consistent. In essence, data lived in one location and other models would point to that location. In this sense, MBSE helped propagate data changes throughout all models that relied on such data. However, a single source of truth is just one solution to the consistency and relevance issue, and comes with some drawbacks, particularly in terms of vulnerability and efficiency. Lately, the concept has morphed into the more general ASOT and authoritative data, where the key is not on the *uniqueness* of the location but on the *certification* of the data being used and/or sourced. MBSE in this sense enables the identification and tagging of data used across the modeling environment.

This idea of an ASOT is the enabler for Digital Engineering, Digital Thread, Digital Twin, and all related digital transformation

artifacts. If the systems engineering team is the technical connective tissue that binds together the project team, MBSE is the digital connective tissue that enables Digital Engineering, which enables creating a Digital Thread, which allows for development of a Digital Twin. Particularly, one could conceive MBSE as the subset of DE that allows all the disparate domains involved in the engineering process to work collectively with that authoritative source of data [8].

Beyond these benefits, MBSE done well enables novel capabilities that accelerate and advance the greater systems lifecycle. At the heart of each of these is representing data and knowledge in a structure and manner that is computable by both human and machine. Freeing knowledge from documents and artifacts, MBSE enables better alignment across the enterprise providing the right data to the right place at the right time at the right level of abstraction in full context presented properly for the consumer to better understand, analyze, and decide. Contrast this with traditional methods that represent one set of data (likely both missing information and including superfluous data for the decision at hand) in a single presentation and lacking context. This dynamic query and presentation of information on demand ensures consistency with the underlying model [10-12]. Properly coupled with strong visualizations including documents, diagrams, tables, and modern options such as dynamic animations and Unreal gaming engines, MBSE unlocks the power of multiple perspectives from engineer to operator to subject matter expert throughout the lifecycle (ref. Figure 6).

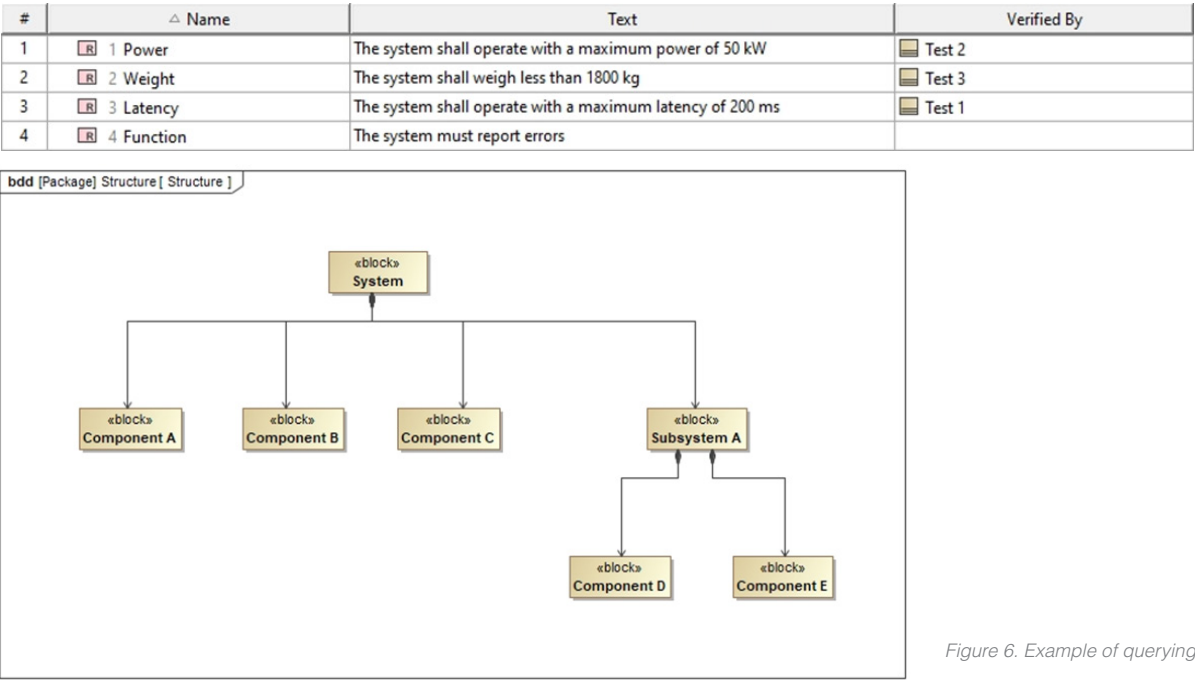


Figure 6. Example of querying a model

Done well, MBSE reflects not only the individual bits of data but also the relationships and dependencies between them helping to move from data to information to knowledge representation. This knowledge map enables classic traceability analysis, such as checking that all requirements are satisfied in the solution and that all functions are allocated to the physical architecture (ref. Figure 7). Moreover, it supports the rapid evaluation of change enabling the

engineering team to trace the impact of a proposed change (be that a new requirement or a new component), identify the affected aspects of the solution architecture, and evaluate the change in the context of previous design decisions (ref. Figure 8). While humans are still responsible for the engineering and analysis, the scoping and context provided by the knowledge map improves the quality and accelerates the analysis.

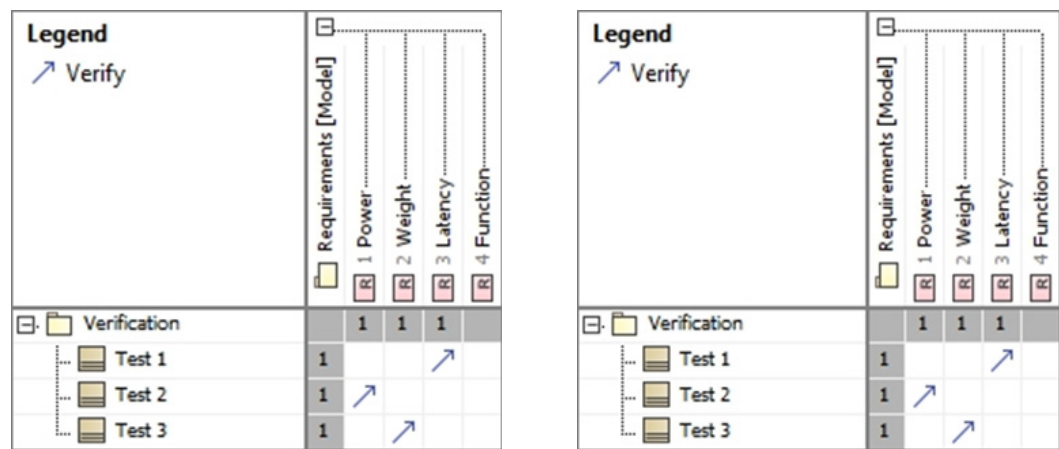


Figure 7. Examples of traceability analysis visualization

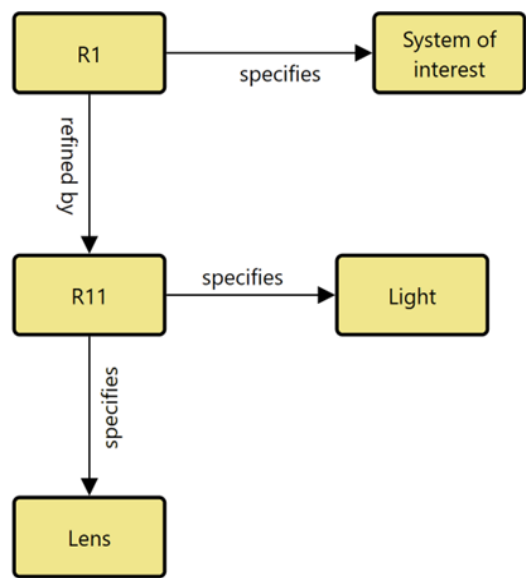


Figure 8. Example of a change propagation analysis visualization.
(The diagram is automatically generated when requirement R1 is flagged because of change. It identifies all elements in the model that are associated with such a requirement.)

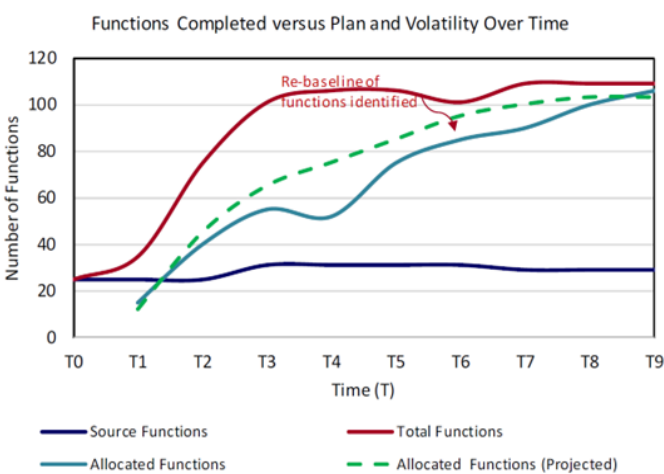


Figure 9. Example of completeness metrics visualization

This knowledge map generated through MBSE can be analyzed through computing techniques to identify completeness and design integrity issues. For example, traditional completeness checks such as ensuring requirements trace to solution elements, requirements are verified, and functions are allocated can be performed consistently and rapidly generating metrics to reflect the maturity of the design (ref. Figure 9). More sophisticated checks can identify design integrity issues such as accounting for all inputs, outputs, and interfaces during decomposition or ensuring interfaces of the right type transfer exchanges between components (ref. Figure 10). Moving beyond traditional computing techniques, machine learning algorithms could be applied to identify patterns and suggest design alternatives.

Best of all, the system model represents a virtual system prototype from day one, albeit at a high level of abstraction early in the project. The model can be dynamically simulated to identify issues and confirm system performance. As the system model is refined and analytic models are coupled with the descriptive architectural model, the level of detail and precision increases. This enables continuous evaluation and verification of the design, accelerating defect detection and enabling the engineering team to rapidly conduct trade studies.

These examples are not exhaustive, but they should give a glimpse of the state of the possible when systems engineering is moved into a modeling environment.

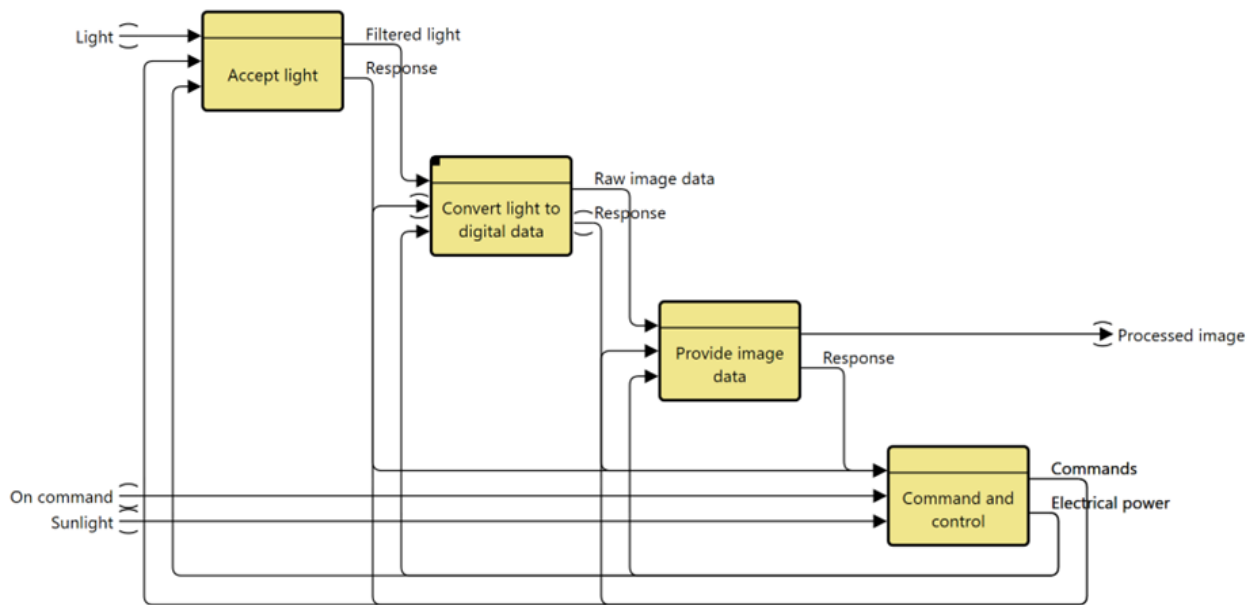


Figure 10. Example of automated design integrity check visualization (Note: Parentheses next to a signal on the sides of the diagram mean that such a signal is missing at the higher level of encapsulation; Parentheses on a signal next to a box indicate that such a signal is not allocated on the lower level of encapsulation.)

Digital Engineering: an integrated digital approach that uses authoritative sources of systems' data and models as a continuum across disciplines to support lifecycle activities from concept to disposal.

Digital Thread: the communication framework that allows connected data flow and integrated view of an asset's data throughout its life cycle across traditionally siloed functional perspective.

Digital Twin: a computational model of a particular physical system with bidirectional communication with its physical counterpart. A digital twin coevolves with the physical system and reflects the state, status, and history of the system.

6. ADOPTING MBSE

At the time of writing this chapter, there is a growing number of organizations adopting and implementing MBSE around the world. However, organizations hold differing views of what MBSE is [13]. In some organizations, MBSE is defined in terms of the tools. Others define MBSE in terms of the models, model artifacts, methods, or processes they use. Some organizations even gloss over the ‘SE’ part of MBSE. Having the correct expectation of what MBSE is and what it can provide for an organization is critical for setting up an adoption effort to be successful in an organization [14].

As has been discussed throughout this chapter, it is critical to know that MBSE is *just* a different way of performing systems engineering. Therefore, when adopting MBSE, it is important to start from processes and practices and not from the tool to be used. The organization must focus on selecting where in their organization MBSE would be the most beneficial with the identification of business value, not simply technical benefit.

Lessons learned indicate that it is easier to adopt MBSE if the scope is limited at the beginning, as an early demonstration of benefits can be shown. In fact, improved organizational outcomes is central to adoption (ref. Figure 11). This comes down to a common element of change management: people want to know that something is going to help them before they commit to learning, applying, and supporting it.

Managers often sidestep this issue when adopting MBSE because it is difficult to show quantitative benefits [8]. However, it is still possible to rely on the anecdotal observations of others that have attempted to implement MBSE and learn from their shared experiences. Successful implementation and adoption seems to require a holistic approach, intentionally targeting the different aspects listed in Table 1 [13]. For example, organizational units that exhibited higher degrees of interconnectedness, standardization, and flexibility reported improved outcomes for MBSE adoption and implementation with respect to exhibited lesser degrees [16].

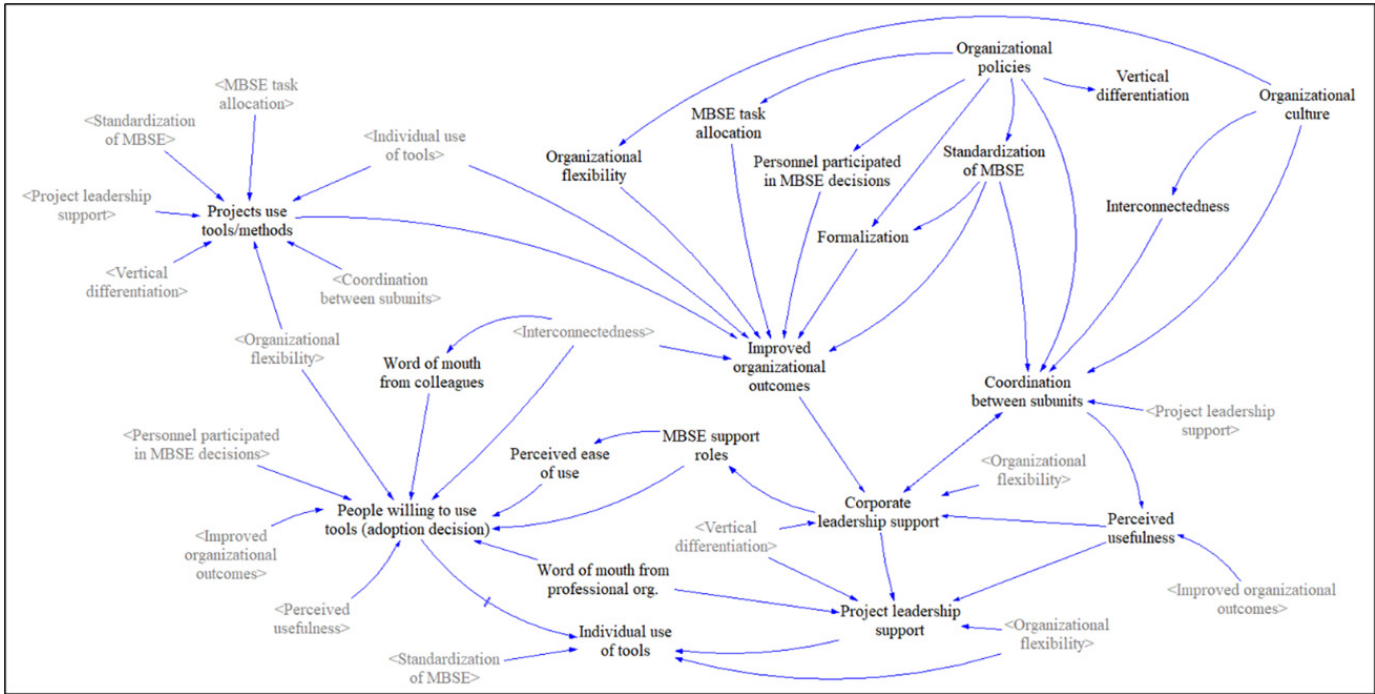


Figure 11. MBSE adoption causal model [from [15]]

Organizational design	Organizational enablers/barriers	Organizational change management
Workforce knowledge/ skills	Leadership/ management support/ commitment	Application of MBSE methods/ processes and modeling practices
Integration	Training	Adoption/ implementation strategy and design
Demonstrated benefits/ results	Resources for implementation	Culture change management
Organizational structure	Tool infrastructure	Willingness to use tools (employee and stakeholder buy-in)

Table 1. Aspects necessary for successful MBSE implementation and adoption [adapted from [13]]

Interconnectedness: people within the organization interact often and are willing to assist others with problems.

Standardization: the use of MBSE tools and methods are standardized across the organizational unit.

Flexibility: organizations can adapt easily to change, new technologies, and processes.

These factors describe some ways an organization can set itself up for success when adopting MBSE. It is important to have some type of network established for people who are learning MBSE. This can take many forms: mentors, coaches, defined experts someone can turn to, or a network of experienced peers. It is also beneficial to have what tools and methods are used standardized across the organizational unit. This can make it easier for informal/formal networks of people to help everyone else who is learning. For example, if the same tool is used across the team, it is more likely that people will be able to help if a problem related to the tool arises. The connection with flexibility highlights the importance of change management. While this factor is more challenging to enact in a short period, there are steps can be taken to make sure people in the organization are prepared to adopt MBSE. Making sure the purpose of MBSE is clear and explaining how it can be beneficial to someone's daily work goes a long way towards having a workforce ready to accept MBSE.

While this type of research on MBSE adoption is helpful, it is important to know that there is no "one size fits all" approach to successfully adopt MBSE. As discussed with standards and methodologies, an organization's purpose behind using MBSE can vary widely. An adoption strategy needs to be in-line with that purpose, along with several other organizational factors (e.g. leadership buy-in, financial support, subject matter expertise of employees, etc.).

There are three main groups within an organization to consider when defining an adoption strategy [17]:

- 1. The initiators and drivers of MBSE.** It is necessary to have some level of knowledge and expertise in the workforce to really drive the effective use of MBSE in projects. There are many modeling pitfalls that are easy to fall into, which could result in the creation of models that do not actually add value to the organization. This is a surefire way to make adoption of MBSE even more difficult. It is important (especially early on) to show the benefits of modelling, so it is key to have the right people on the team supporting the effort.
- 2. The organizational units that should work model-based.** Building models is great, but if the models are not being used for anything then what is the point? For MBSE to fulfill its intended purpose, it needs to fit in an organization's process. In some cases, this could take adjusting workflows, job descriptions/responsibilities, or even the structure of the organization. It is more than simply translating document-centric approaches into a model-based world; it is a digital *transformation*. An organizational unit adopting MBSE needs to conscientiously plan out where in its process it makes sense to use MBSE to create the most value.
- 3. The organization units responsible for the time and budget of the engineering projects.** MBSE represents a significant investment in money and time for an organization. This fact needs to be considered and used to manage expectations for leadership and others. The organizational unit that is adopting MBSE needs to be given the space and resources to do it successfully.